Part I — Developer user manual

1. Prerequisites

Before installing and running Claude Code you need:

- **Git for Windows — required** (Windows laptops). Claude Code's shell tooling runs through Git's `bash.exe`, and its version-control features (diffs, commits, branches) use `git` itself. A standard install is found automatically (Claude Code looks in `C:\Program Files\Git\bin\bash.exe` and `C:\Program Files (x86)\Git\bin\bash.exe`). If your Git lives anywhere else — a per-user or portable install — set the `CLAUDE_CODE_GIT_BASH_PATH` environment variable to your `bash.exe` location (see §5.1).
- **On Linux:** a glibc x86-64 distribution (Ubuntu/Debian/RHEL and derivatives — the portal serves the `linux-x64` glibc build, not the Alpine/musl one), with `git`, `bash`, and the `sha256sum` coreutils (present on any standard developer distribution); `python3` is used by the installer to write your settings file — without it the binary still installs and the installer prints the settings to add by hand.
- **Node.js / npm — NOT required.** This deployment distributes the precompiled native binary (`claude.exe` on Windows, `claude` on Linux); nothing is installed from npm, and `npm install`-based instructions you may find online do not apply here (updates are locked down and this network does not reach npm anyway).
- **An Okta account** in the authorized group, with MFA enrolled — the same corporate SSO sign-in gates both the download portal and Claude Code itself.
- **The managed login policy on your machine.** IT delivers it automatically by GPO/MDM; there is nothing for you to do. Without it Claude Code has no gateway login option at all — if `/login` shows no "Cloud gateway" screen, contact IT rather than trying to configure it yourself (a developer with local admin can self-serve it — §8).
- **The gateway certificate fingerprint published by IT** — you confirm it once, at first connect. The download portal also shows the current value live on its **Gateway fingerprint** page (§4.4).
- **No admin rights** — the install itself is entirely user-scope.

2. Installing

2.1 Downloading from the portal

The installer comes from the **download portal**: open the portal link IT published (`https://<gateway-host>/portal`) in your browser and sign in with your normal Okta SSO. The portal opens on a home page of cards — the portal is also where you can check your usage and quotas, read this guide, and see the gateway fingerprint (§4) — pick **Download installer**. Before the download, the page asks for three picks:

1. **Cost center** — choose yours first.
2. **Team** — the team list fills in with only the teams belonging to the cost center you picked. Choose the team you actually work in.
3. **Platform** — **Windows (64-bit)** or **Linux (x64)**, then start the download.

(If your browser has scripts disabled, the same page works as two quick page loads instead: pick the cost center, continue, then pick the team and platform.)

What the picks are for, and how to choose. Your selections are baked into the installer and become labels on your usage telemetry (`cost_center=…`, `team=…`), so your Claude Code usage and spend are attributed to the right team and cost center on IT's dashboards. They are **attribution only** — they grant nothing and restrict nothing: no permissions, models, or limits depend on them. Choose the team you actually work in; if you don't know which cost center funds your work, ask your manager or IT rather than picking the closest match — a wrong pick means your usage shows up under someone else's budget. Both lists (and which teams belong to which cost center) are maintained by IT, so if yours is missing, tell IT — don't substitute another. Picked wrong? Download again with the right pair and re-run the installer: re-installing is harmless and simply overwrites the labels. Each download is recorded (who, which team/cost center, which version) in an audit log.

2.2 Running the installer (Windows)

The ZIP contains `claude.exe`, the installer script (`Install-ClaudeCode.ps1`), an `install.cmd` with your portal picks and the deployment's standard options baked in, a `README.txt`, and — when the deployment bundles one — an `extra-ca.pem` trust file. Extract it and double-click `install.cmd`. No elevation prompt appears; the installer does exactly three things, all in your own profile:

- **Binary** → `%USERPROFILE%\.local\bin\claude.exe`, verified (SHA-256 against the release manifest + Anthropic Authenticode) on a local staging copy before it is moved into place.
- **User PATH** → `%USERPROFILE%\.local\bin` is appended to the *user* `Path` environment variable (registry-backed, persists; no machine PATH edit).
- **User configuration** → an `env` block merged into `%USERPROFILE%\.claude\settings.json` (your own settings file). The merge preserves every existing top-level key and every unrelated `env` key, and refuses to overwrite a file it cannot parse. The keys written:

Key (under `env`)	Set by	Purpose
`DISABLE_UPDATES = 1`	`-DisableUpdates`	Blocks all update paths (background checks **and** manual `claude update` / `claude install`) — keeps users on the distributed build
`DISABLE_AUTOUPDATER = 1`	`-DisableUpdates`	Background-check lockdown, defense in depth
`OTEL_RESOURCE_ATTRIBUTES`	`-Team` / `-CostCenter` (your portal picks, passed via `install.cmd`)	Telemetry grouping labels (`team=…,cost_center=…`); telemetry itself is enabled centrally by the gateway
`NODE_EXTRA_CA_CERTS`	`-ExtraCaCertPath`	Enterprise CA trust for the gateway TLS chain (the precompiled binary honors it)

These are ordinary environment variables, honored from the user settings file — **not** policy keys. The installer never writes `%ProgramFiles%\ClaudeCode\managed-settings.json` and never touches `HKx\SOFTWARE\Policies\ClaudeCode`. A SYSTEM-context run is refused outright.

After installing, **open a new terminal** so the updated PATH is picked up.

2.3 Running the installer (Linux)

The Linux ZIP contains the `claude` binary (linux-x64), the installer script (`install-claude-code.sh`), an `install.sh` with your portal picks and the deployment's standard options baked in, a `README.txt`, and — when the deployment bundles one — an `extra-ca.pem` trust file. Unzip it and run:

```
unzip claude-code-<version>-linux.zip -d claude-code && cd claude-code
bash install.sh
```

No root or sudo is involved; the installer is the Linux twin of the Windows one and does the same three user-scope things:

- **Binary** → `~/.local/bin/claude` (mode 0755), verified by SHA-256 against the release manifest on a local staging copy before it is moved into place. (There is no Authenticode on Linux; the checksum chain back to the GPG-verified release manifest is the integrity check.)
- **PATH** → if `~/.local/bin` is not already on your `PATH` (most distributions put it there), a guarded `export PATH=…` line is appended to `~/.bashrc`. If your shell is not bash, add the equivalent line to your shell's profile yourself.
- **User configuration** → the same `env` block as §2.2's table, merged into `~/.claude/settings.json` with the same rules (preserves unrelated keys, refuses to overwrite a file it cannot parse). The one path difference: a bundled enterprise CA is copied to `~/.local/bin/claude-extra-ca.pem` and referenced from `NODE_EXTRA_CA_CERTS`.

The installer refuses to run as root (it would install into root's home, not yours). After installing, **open a new terminal** and continue with §3 — the sign-in flow is the same on Linux, and the login policy your machine needs is the Linux managed-settings file (§8.5), delivered by IT.

3. First launch & signing in

Signing in is effectively a one-time step: the session persists and refreshes itself. The login method is locked to the corporate gateway and the URL is pre-filled — you never choose an account type or type a URL.

1. Open a **new** terminal and run `claude`.
2. Two things can happen here, **both normal**:
3. Claude Code opens directly on the **gateway login screen**, or
4. it drops you into the **chat window** with a notice telling you to run `/login`. Type `/login` and the same gateway screen appears.
5. The gateway screen shows the pre-filled gateway URL. Press **Enter** to accept and connect.
6. Claude Code usually does **not** launch your browser by itself. Copy the authentication URL it prints and paste it into your browser, then complete the Okta sign-in (+ MFA).
7. At first connect Claude Code also shows the gateway certificate fingerprint and asks you to confirm it (trust-on-first-use). Compare it against the fingerprint IT published — the portal's **Gateway fingerprint** page (§4.4) shows the current value. Do not blind-accept it; that prompt is what detects TLS interception in front of the gateway.

8. Back in the terminal, Claude Code tells you to press Enter to continue. About half the time the Enter key does nothing at this point — just **close the command window, open a new one, and run** `claude`: you will be fully signed in.
9. Occasionally Claude Code comes back from a successful sign-in not responding to the keyboard at all. Close it and reopen it — the session is already saved.
10. If you have used Claude in some other form before (claude.ai, a personal API key, an earlier install), you may see a **yellow warning** that a previous model was not found and that you are now using **Opus 4.8**. This is normal — see §5.7.

Signing out (if you ever need to): use `claude auth logout` — never the `/logout` slash command. On this network `/logout` leaves Claude Code unable to start (§5.6 has the recovery).

4. Day-to-day notes

4.1 Everyday behavior

- `/model` lists exactly three models — Opus 4.8 (the default), Sonnet 5, and Sonnet 4.5. That is by design, not an error; nothing else is served here.
- Web tools (WebFetch / WebSearch) and MCP tools are disabled centrally and cannot be re-enabled in your local settings.
- Updates are disabled; you always run the IT-distributed build, so `claude update` doing nothing is intentional. New versions are published to the download portal (the version is shown on its page) — when IT announces one, download a fresh installer ZIP and run `install.cmd` again.
- `/status` shows which configuration sources are active — useful to see what is centrally managed vs. your own `settings.json`.

The download portal (`https://<gateway-host>/portal`, same Okta sign-in as the installer download) is also your self-service window into the deployment — the three pages below.

4.2 My usage & quotas

The portal's **My usage & quotas** page shows your own Claude Code spend: for each budget period that applies to you (daily, weekly, or monthly) it lists your spend cap, your spend so far this period, and the percentage used, with a colored progress bar. It also tells you where each cap comes from — a cap set for you personally, one inherited from an Okta group you are in, or the organization-wide cap. If no cap applies to you, the page still shows your spend, marked "no cap".

Notes:

- Hitting a cap does not break anything permanently — Claude Code starts refusing requests with a "spend limit reached" message until the period rolls over or an admin raises the cap. This page is how you see the cap coming before you hit it.
- Spend figures come from the gateway's own metering, the same numbers cap enforcement uses.
- If the page says usage display is not enabled on this deployment, that is a deployment-side setting — mention it to IT if you expected to see your numbers.

4.3 The user guide

The portal's **User guide (PDF)** page shows this manual: read it in the browser or use its download link to save a copy. The portal always serves the version IT most recently published, so prefer it over stale local copies.

4.4 Gateway fingerprint

The portal's **Gateway fingerprint** page shows the gateway's current TLS certificate fingerprint, fetched live from the gateway itself. It is printed in the two common formats — uppercase pairs separated by colons (`AA:BB:…`) and the same value as one lowercase hex string — so whichever style Claude Code's first-connect prompt uses, you can compare directly (§3 step 5).

If the fingerprint Claude Code shows you matches neither form, **stop — do not accept it** — and contact IT: a mismatch can mean something is intercepting TLS between your machine and the gateway. Also expect the value to change when IT rotates the gateway certificate; the portal page always shows the current one.

5. Troubleshooting

5.1 Claude Code can't find Git / bash

Symptoms: a startup error saying Claude Code requires Git for Windows, shell commands failing to run, or *"Claude Code was unable to find CLAUDE_CODE_GIT_BASH_PATH path …"*.

- If Git for Windows is not installed, install it (no admin needed for the per-user installer).
- If Git is installed somewhere other than the standard `C:\Program Files\Git` location (per-user install, portable Git), point Claude Code at your `bash.exe` with the `CLAUDE_CODE_GIT_BASH_PATH` environment variable:

```
powershell setx CLAUDE_CODE_GIT_BASH_PATH "D:\Tools\Git\bin\bash.exe"
```

then open a **new** terminal. Alternatively, put it in the `env` block of `%USERPROFILE%\.claude\settings.json` alongside the installer's keys:

```
json { "env": { "CLAUDE_CODE_GIT_BASH_PATH": "D:\\Tools\\Git\\bin\\bash.exe" } }
```

The value must be the full path to `bash.exe` **itself** (normally `…\Git\bin\bash.exe`), not the Git folder and not `git.exe`.

5.2 No login screen at launch — dropped into the chat window

Sometimes the first launch does not take you to the login screen and instead opens the chat window with a notice to log in. This is normal: run `/login` and continue from §3 step 3.

If `/login` shows **no gateway screen at all** (an account picker with no "Cloud gateway" option), the managed login policy is missing from the machine — contact IT (§8; there is nothing you can set in user scope to fix this).

5.3 The browser never opens during sign-in

Expected most of the time: after you accept the gateway screen, Claude Code typically does not launch the browser itself. Copy the authentication URL from the terminal and paste it into your browser manually.

5.4 "Press Enter to continue" does nothing

After the browser sign-in completes, roughly half the time the terminal's Enter key has no effect on the "press Enter to continue" prompt. Close the command window, open a new one, and run `claude` — you will be fully signed in (the session was already saved).

5.5 Keyboard input not recognized after signing in

Occasionally, after a successful sign-in, Claude Code stops responding to keyboard input entirely. Close Claude Code and reopen it; the session persists, so you will not have to sign in again.

5.6 Startup fails: "Unable to connect to Anthropic services"

Claude Code exits at launch before any login screen appears. This happens when the onboarding flag is unset — on a first-ever run on a clean profile, or after running the `/logout` slash command — because a not-yet-onboarded client tries to reach Anthropic's public endpoints, which this network blocks by design.

Fix (no admin needed): mark onboarding as completed in `%USERPROFILE%\.claude.json`:

```
$p = "$env:USERPROFILE\.claude.json"
Copy-Item $p "$p.bak"          # this file also holds project history — back it up
$j = Get-Content $p -Raw | ConvertFrom-Json
if ($j.PSObject.Properties.Name -contains 'hasCompletedOnboarding') {
  $j.hasCompletedOnboarding = $true
} else {
  $j | Add-Member -NotePropertyName hasCompletedOnboarding -NotePropertyValue $true
}
$j | ConvertTo-Json -Depth 100 | Set-Content $p -Encoding utf8
```

(If `%USERPROFILE%\.claude.json` does not exist yet, create it containing just `{"hasCompletedOnboarding": true}`.)

On Linux the same flag lives in `~/.claude.json`; back the file up (`cp ~/.claude.json ~/.claude.json.bak`) and set it with:

```
python3 - <<'EOF'
import json, os
p = os.path.expanduser("~/.claude.json")
d = json.load(open(p)) if os.path.exists(p) else {}
d["hasCompletedOnboarding"] = True
json.dump(d, open(p, "w"), indent=2)
EOF
```

Then open a **new** terminal, run `claude`, and run `/login`. Because `/logout` also deletes the pinned certificate fingerprint, the fingerprint confirmation from §3 step 5 will come back — re-check it against the IT-published value. To avoid the whole trap, sign out with `claude auth logout`, never `/logout`. The service-desk version of this procedure is `om-runbooks.md` runbook 12.

5.7 Yellow warning: a model was not found — now using Opus 4.8

If you previously used Claude in some other form (claude.ai, a personal API key, another workplace's install), your old settings may name a model this gateway does not serve. On login Claude Code warns in yellow that the model was not found and switches you to **Opus 4.8**. This is normal behavior, not an error — Opus 4.8 is the default here, and `/model` shows what you can switch to.

5.8 Startup fails: your Claude Code version is below the minimum

The gateway enforces a minimum Claude Code version (normally the gateway's own version). If your installed client is older, Claude Code exits at startup with a message saying the version is below the required minimum. The built-in updater is disabled on this deployment, so the fix is to download and run the latest installer from the portal (§2.1) — no admin rights needed, same as the first install. Your sign-in, settings, and history are kept. You'll hit this after the platform team upgrades the gateway; the download portal always has the matching installer.

5.9 Bounced to the browser sign-in every hour

If you are pushed through the browser SSO at every session expiry (about hourly) and `/login` then shows the default picker until you restart Claude Code, report it to your admins — it means the Okta app is missing the **Refresh Token** grant type (so the session cannot refresh silently). Admin fix: `okta-request-email.md` and `om-runbooks.md`. It is not a problem with your machine or settings.

This manual is Part I (§1–§5) of `docs/operations/client-config.md`. References to §6–§9 point at Part II — Administrators — in the full document; end users normally don't need it (§8 self-service requires local admin).